

CO3 ASSESSMENT TOOL 1 – Analytical Problem Solving

**SIMATS**
ENGINEERING

SIMATS Online Programming Lab
C PROGRAMMING

RESHINI PRIYA B
10/10/2025

CO3 - AT1 - Analytical Problem solving

QUESTIONS

Q1
CO3 - AT1 - Analytical Problem Solving
20 marks

Q2
CO3 - AT1 - Analytical Problem Solving
20 marks

Q3
CO3 - AT1 - Analytical Problem Solving
20 marks

Q4
CO3 - AT1 - Analytical Problem Solving
20 marks

Q5
CO3 - AT1 - Analytical Problem Solving
20 marks

5/5 answered

Q1 CO3 - AT1 - Analytical Problem Solving

20 marks

1. Given an $N \times N$ matrix, Write a C Program to determine whether it is:
a) Symmetric
b) Upper triangular
c) Lower triangular
d) Diagonal matrix
e) Identity matrix
f) Display all applicable classifications.

Your Code

```
1 #include <stdio.h>
2
3 int main() {
4     int n, i, j;
5     int a[20][20];
6
7     printf("Enter size of matrix (N x N): ");
8     scanf("%d", &n);
9
10    printf("Enter elements of matrix:\n");
11    for (i = 0; i < n; i++) {
12        for (j = 0; j < n; j++) {
13            scanf("%d", &a[i][j]);
14        }
15    }
16
17    int isSymmetric = 1, isUpper = 1, isLower = 1, isDiagonal = 1, isIdentity = 1;
18
19    for (i = 0; i < n; i++) {
20        for (j = 0; j < n; j++) {
21            // Symmetric check
22            if (a[i][j] != a[j][i]) {
23                isSymmetric = 0;
24            }
25
26            // Upper triangular check (elements below diagonal must be 0)
27            if (i < j && a[i][j] != 0) {
28                isUpper = 0;
29            }
30
31            // Lower triangular check (elements above diagonal must be 0)
32            if (i > j && a[i][j] != 0) {
33                isLower = 0;
34            }
35
36            // Diagonal check (all non-diagonal elements must be 0)
37            if (i != j && a[i][j] != 0) {
38                isDiagonal = 0;
39            }
40
41            // Identity check (diagonal elements must be 1, others 0)
42            if (i == j && a[i][j] != 1) {
43                isIdentity = 0;
44            }
45            if (i != j && a[i][j] != 0) {
46                isIdentity = 0;
47            }
48        }
49    }
50
51    printf("\nMatrix Classification Results:\n");
52
53    int anyMatch = 0;
54
55    if (isSymmetric) {
56        printf("The matrix is Symmetric.\n");
57        anyMatch = 1;
58    }
59    if (isUpper) {
60        printf("The matrix is Upper Triangular.\n");
61        anyMatch = 1;
62    }
63    if (isLower) {
64        printf("The matrix is Lower Triangular.\n");
65        anyMatch = 1;
66    }
67    if (isDiagonal) {
68        printf("The matrix is Diagonal.\n");
69        anyMatch = 1;
70    }
71    if (isIdentity) {
72        printf("The matrix is an Identity Matrix.\n");
73        anyMatch = 1;
74    }
75
76    if (!anyMatch) {
77        printf("The matrix does not satisfy any special classification.\n");
78    }
79
80    return 0;
81 }
```

Run Save

Input

```
3
1 0 1
0 1 0
0 0 1
```

Output

© 2025 SIMATS Online Lab. All rights reserved.

C03 - AT1 - Analytical Problem solving

← Back

QUESTIONS

Q1
C03 - AT1 - Analytical Problem Solving
20 marks**Q2**
C03 - AT1 - Analytical Problem Solving
20 marks**Q3**
C03 - AT1 - Analytical Problem Solving
20 marks**Q4**
C03 - AT1 - Analytical Problem Solving
20 marks**Q5**
C03 - AT1 - Analytical Problem Solving
20 marks

5/5 answered

Q2 C03 - AT1 - Analytical Problem Solving

20 marks

2. Given an array of integers, write a C program that:
- Determines whether the array is sorted.
 - If sorted, performs Binary Search.
 - Otherwise, performs Linear Search.
 - Displays the search method used and the number of comparisons performed.

Your Code

```
#include <stdio.h>

1
2
3
4 int isSorted(int arr[], int n) {
5     for (int i = 0; i < n - 1; i++) {
6         if (arr[i] > arr[i + 1])
7             return 0; // not sorted
8     }
9     return 1; // sorted
10 }
11
12 // Linear Search Function
13 int linearSearch(int arr[], int n, int key, int *comparisons) {
14     *comparisons = 0;
15     for (int i = 0; i < n; i++) {
16         (*comparisons)++;
17         if (arr[i] == key)
18             return i;
19     }
20     return -1;
21 }
22
23 // Binary Search Function
24 int binarySearch(int arr[], int n, int key, int *comparisons) {
25     int low = 0, high = n - 1;
26     *comparisons = 0;
27
28     while (low <= high) {
29         int mid = (low + high) / 2;
30         (*comparisons)++;
31
32         if (arr[mid] == key)
33             return mid;
34         else if (arr[mid] < key)
35             low = mid + 1;
36         else
37             high = mid - 1;
38     }
39     return -1;
40 }
41
42 int main() {
43     int n, key, comparisons = 0, result;
44     printf("Enter number of elements: ");
45     scanf("%d", &n);
46
47     int arr[n];
48     printf("Enter %d elements: ", n);
49     for (int i = 0; i < n; i++)
50         scanf("%d", &arr[i]);
51
52     printf("Enter the element to search: ");
53     scanf("%d", &key);
54
55     if (isSorted(arr, n)) {
56         printf("\nArray is sorted.\n");
57         printf("Using Binary Search...\n");
58         result = binarySearch(arr, n, key, &comparisons);
59     } else {
60         printf("\nArray is not sorted.\n");
61         printf("Using Linear Search...\n");
62         result = linearSearch(arr, n, key, &comparisons);
63     }
64
65     if (result != -1)
66         printf("Element found at index %d.\n", result);
67     else
68         printf("Element not found.\n");
69
70     printf("Number of comparisons made: %d\n", comparisons);
71     return 0;
72 }
```

Run

Save

Input

```
5
2 3 5 7 4
4
```

Output

CO3 - AT1 - Analytical Problem solving

← Back

QUESTIONS

Q1
CO3 - AT1 - Analytical Problem Solving
20 marks**Q2**
CO3 - AT1 - Analytical Problem Solving
20 marks**Q3**
CO3 - AT1 - Analytical Problem Solving
20 marks**Q4**
CO3 - AT1 - Analytical Problem Solving
20 marks**Q5**
CO3 - AT1 - Analytical Problem Solving
20 marks

3/5 answered

Q3 CO3 - AT1 - Analytical Problem Solving

20 marks

3. Store marks of N students in M subjects using a two-dimensional array. Generate a report containing:
- Student topper
 - Subject topper
 - Class average
 - Subject-wise average
 - Students scoring above the overall average
 - Rank list without modifying the original data

Your Code

```
1 #include <stdio.h>
2
3 int main() {
4     int n, m;
5     printf("Enter number of students: ");
6     scanf("%d", &n);
7     printf("Enter number of subjects: ");
8     scanf("%d", &m);
9
10    int marks[n][m];
11    int studentTotal[n];
12    int subjectTotal[m];
13    float rank[n];
14
15    for (int j = 0; j < m; j++)
16        subjectTotal[j] = 0;
17
18    // Input marks
19    for (int i = 0; i < n; i++) {
20        studentTotal[i] = 0;
21        printf("\nEnter marks for Student %d in %d subjects:\n", i + 1, m);
22        for (int j = 0; j < m; j++) {
23            printf("Subject %d: ", j + 1);
24            scanf("%d", &marks[i][j]);
25            studentTotal[i] += marks[i][j];
26            subjectTotal[j] += marks[i][j];
27        }
28    }
29
30    // a) Student topper
31    int topStudent = 0;
32    for (int i = 1; i < n; i++)
33        if (studentTotal[i] > studentTotal[topStudent])
34            topStudent = i;
35
36    // b) Subject topper (highest scorer in each subject)
37    int subjectTopper[m];
38    for (int j = 0; j < m; j++) {
39        subjectTopper[j] = 0;
40        for (int i = 1; i < n; i++)
41            if (marks[i][j] > marks[subjectTopper[j]][j])
42                subjectTopper[j] = i;
43    }
44
45    // c) Class Average (average of all marks)
46    int grandTotal = 0;
47    for (int i = 0; i < n; i++)
48        grandTotal += studentTotal[i];
49    float classAverage = (float) grandTotal / (n * m);
50
51    // d) Subject-wise Average
52    float subjectAverage[m];
53    for (int j = 0; j < m; j++)
54        subjectAverage[j] = (float) subjectTotal[j] / n;
55
56    // e) Overall average per student, students scoring above it
57    float overallAvgPerStudent = (float) grandTotal / n;
58
59    // f) Rank list without modifying original data
60    // Copy studentTotal into a separate array for ranking
61    int rankOrder[n];
62    for (int i = 0; i < n; i++)
63        rankOrder[i] = i;
64
65    // Simple bubble sort on indices based on studentTotal (original)
66    for (int i = 0; i < n - 1; i++) {
67        for (int j = 0; j < n - i - 1; j++) {
68            if (studentTotal[rankOrder[j]] < studentTotal[rankOrder[j + 1]]) {
69                int temp = rankOrder[j];
70                rankOrder[j] = rankOrder[j + 1];
71                rankOrder[j + 1] = temp;
72            }
73        }
74    }
75
76    // ===== REPORT =====
77    printf("\n===== STUDENT MARKS REPORT =====\n");
78
79    printf("\na) Student Topper: Student %d with Total = %d\n",
80        topStudent + 1, studentTotal[topStudent]);
81
82    printf("\nb) Subject Toppers:\n");
83    for (int j = 0; j < m; j++)
84        printf("    Subject %d -> Student %d (Marks = %d)\n",
85            j + 1, subjectTopper[j] + 1, marks[subjectTopper[j]][j]);
86
87    printf("\nc) Class Average (overall): %.2f\n", classAverage);
88
89    printf("\nd) Subject-wise Average:\n");
90    for (int j = 0; j < m; j++)
91        printf("    Subject %d -> %.2f\n", j + 1, subjectAverage[j]);
92
93    printf("\ne) Students scoring above overall average (%.2f):\n", overallAvgPerStudent);
94    for (int i = 0; i < n; i++)
95        if (studentTotal[i] > overallAvgPerStudent)
96            printf("    Student %d (Total = %d)\n", i + 1, studentTotal[i]);
97
98    printf("\nf) Rank List (without modifying original data):\n");
99    for (int i = 0; i < n; i++)
100        printf("    Rank %d -> Student %d (Total = %d)\n",
101            i + 1, rankOrder[i] + 1, studentTotal[rankOrder[i]]);
102
103    return 0;
104 }
```

Run Save

Input

```
2
4
5 6
```

Output

CO3 - AT1 - Analytical Problem solving

← Back

QUESTIONS

Q1
CO3 - AT1 - Analytical Problem Solving
20 marks**Q2**
CO3 - AT1 - Analytical Problem Solving
20 marks**Q3**
CO3 - AT1 - Analytical Problem Solving
20 marks**Q4**
CO3 - AT1 - Analytical Problem Solving
20 marks**Q5**
CO3 - AT1 - Analytical Problem Solving
20 marks

5/5 answered

Q4 CO3 - AT1 - Analytical Problem Solving

20 marks

4. Given N strings, write a C program to:
- Sort them lexicographically.
 - Find the longest and shortest strings.
 - Count duplicate strings.
 - Display strings beginning with vowels.

Your Code

```
#include <stdio.h>
#include <string.h>
#include <stdlib.h>
#include <ctype.h>

int main() {
    int n, i, j;

    printf("Enter the number of strings: ");
    if (scanf("%d", &n) != 1) return 1;
    getchar(); // Consume newline

    char strn[100];
    char temp[100];

    printf("Enter %d strings:\n", n);
    for (i = 0; i < n; i++) {
        fgets(strn[i], 100, stdin);
        strn[i][strlen(strn[i], "\n")] = 0; // Remove newline character
    }

    // a) Sort them lexicographically (Bubble Sort)
    for (i = 0; i < n - 1; i++) {
        for (j = 0; j < n - i - 1; j++) {
            if (strcmp(strn[j], strn[j + 1]) > 0) {
                strcpy(temp, strn[j]);
                strcpy(strn[j], strn[j + 1]);
                strcpy(strn[j + 1], temp);
            }
        }
    }

    printf("Sorted strings:\n");
    for (i = 0; i < n; i++) {
        printf("%s\n", strn[i]);
    }

    // b) Find the longest and shortest strings
    int longestId = 0, shortestId = 0;
    for (i = 1; i < n; i++) {
        if (strlen(strn[i]) > strlen(strn[longestId])) {
            longestId = i;
        }
        if (strlen(strn[i]) < strlen(strn[shortestId])) {
            shortestId = i;
        }
    }

    printf("Longest string: %s", strn[longestId]);
    printf("Shortest string: %s\n", strn[shortestId]);

    // c) Count duplicate strings
    int duplicateCount = 0;
    // Since the array is sorted, duplicates are adjacent
    for (i = 0; i < n - 1; i++) {
        if (strcmp(strn[i], strn[i + 1]) == 0) {
            duplicateCount++;
            // Skip the rest of the identical strings to count each group
            while (i < n - 1 && strcmp(strn[i], strn[i + 1]) == 0) {
                i++;
            }
        }
    }

    printf("Number of duplicate string groups: %d\n", duplicateCount);

    // d) Display strings beginning with vowels
    printf("Strings beginning with vowels:\n");
    int foundVowel = 0;
    for (i = 0; i < n; i++) {
        char first = tolower(strn[i][0]);
        if (first == 'a' || first == 'e' || first == 'i' || first == 'o' || first == 'u') {
            printf("%s\n", strn[i]);
            foundVowel = 1;
        }
    }

    if (!foundVowel) {
        printf("None\n");
    }

    return 0;
}
```

Input

```
3
5 6 7
4
```

Output

C03 - AT1 - Analytical Problem solving

← Back

QUESTIONS

Q1
C03 - AT1 - Analytical Problem
Solving
20 marks**Q2**
C03 - AT1 - Analytical Problem
Solving
20 marks**Q3**
C03 - AT1 - Analytical Problem
Solving
20 marks**Q4**
C03 - AT1 - Analytical Problem
Solving
20 marks**Q5**
C03 - AT1 - Analytical Problem
Solving
20 marks

3/5 answered

Q5 C03 - AT1 - Analytical Problem Solving

20 marks

5. Accept a string containing repeated characters. Compress it by replacing consecutive repeated characters with the character followed by its count.

Example:

Input:

aaabbbcccccdd

Output:

a3b3c4d2

Also calculate the compression ratio.

Your Code

```
#include <stdio.h>
#include <string.h>

int main() {
    char s[1000];
    char compressed[2000];
    printf("Enter a string: ");
    scanf("%s", s);

    int n = (int)strlen(s);
    int i = 0, k = 0;

    while (i < n) {
        char ch = s[i];
        int count = 1;

        i++;
        while (i < n && s[i] == ch) {
            count++;
            i++;
        }

        compressed[k++] = ch;

        // Add count as digits
        char numStr[20];
        sprintf(numStr, "%d", count);
        int lenNum = (int)strlen(numStr);
        for (int j = 0; j < lenNum; j++) {
            compressed[k++] = numStr[j];
        }

        compressed[k] = '\0';

        int compressedLen = (int)strlen(compressed);
        double ratio = (double)compressedLen / (double)n;

        printf("Compressed Output: %s\n", compressed);
        printf("Original Length: %d\n", n);
        printf("Compressed Length: %d\n", compressedLen);
        printf("Compression Ratio (compressed/original): %.4f\n", ratio);

        return 0;
    }
```

Run

Save

Input

5
6 7 8

Output